

Sharpi Support Automation — Arquitetura MVP

Fluxo ponta a ponta (Sprint 1)

None

flowchart TD

```
A[Mensagem WhatsApp<br/>grupo piloto] -->|Business API webhook| B[POST /webhook/whatsapp<br/>FastAPI + HMAC verify]
B --> C{é suporte?<br/>allowlist + heurística}
C -->|não| X[Drop + log]
C -->|sim| D[Download mídia<br/>se tiver imagem]
D --> E[Triage Agent<br/>Claude Sonnet 4.6<br/>tool-use structured output]

E --> F[Priority Scorer<br/>determinístico]

F --> G1[Encore API<br/>fetch ClientProfile]
F --> G2[Sentry API<br/>erros 24h do componente]
F --> G3{component?}
G3 -->|ai_capture ou<br/>ai_failure| G3A[Braintrust API<br/>traces de IA 2h]
G3 -->|outros| G3B[BetterStack API<br/>logs 1h do componente]
F --> G4[Linear GraphQL<br/>tickets similares]

G1 --> H[Context Enricher<br/>merge em paralelo]
G2 --> H
G3A --> H
G3B --> H
G4 --> H

H --> I{duplicado?<br/>mesma empresa +<br/>componente + aberto < 48h}
I -->|sim| J[Comenta no<br/>ticket existente]
I -->|não| K[Cria issue Linear<br/>categoria flat + labels<br/>+ priority + descrição]

J --> L[Slack #support-triage<br/>notificação duplicata]
K --> L2[Slack #support-triage<br/>novo ticket, canal único<br/>SEM @mention]
L2 --> RET[Time responde cliente<br/>no WhatsApp MANUALMENTE<br/>olhando o ticket]

subgraph "Fase 2 (não implementa agora)"
    M[Ticket fecha no Linear] -. -> N[Webhook /webhook/linear]
    N -. -> O[WhatsApp outbound<br/>confirmação + CSAT]
    O -. -> P[Cliente responde 1-5]
    P -. -> Q[CSAT store]
end

end
```

```

subgraph "Feedback loop (começa Sprint 1)"
  R[Humano reclassifica<br/>no Linear] --> S[Webhook /webhook/linear]
  S --> T[trriage_feedback table<br/>original vs corrigido]
  T -.->|dataset futuro| U[Prompt tuning<br/>evals Braintrust]
end

K --> R

```

Componentes e responsabilidades

None

```

flowchart LR
  subgraph "Ingestion"
    WA[WhatsApp Business API<br/>Meta/Twilio/Z-API]
  end

  subgraph "sharpi-support-automation service"
    WH[webhook_whatsapp.py<br/>HMAC + parse]
    FL[filter.py<br/>allowlist + heurística]
    TA[trriage_agent_v2.py<br/>Claude tool-use<br/>+ component_to_category]
    PS[priority_scorer.py<br/>determinístico]
    CE[context_enricher.py v2<br/>routing por componente]
    BC[braintrust_client.py<br/>traces IA]
    BSC[betterstack_client.py<br/>logs infra]
    LC[linear_client.py<br/>GraphQL]
    SN[slack_notifier.py<br/>webhook canal único]
    FS[feedback_store.py<br/>Postgres/KV]
  end

  subgraph "Serviços externos"
    EN[Encore backend<br/>GET /internal/clients/by-phone]
    SE[Sentry<br/>Issues API]
    BS[BetterStack<br/>Logs API<br/>infra]
    BT[Braintrust<br/>Traces API<br/>IA]
    LI[Linear<br/>GraphQL API]
    SL[Slack<br/>Webhook #support-triage]
    CL[Claude API<br/>Sonnet 4.6]
  end

  WA --> WH
  WH --> FL
  FL --> TA
  TA --> CL
  TA --> PS
  PS --> CE
  CE --> EN

```

```

CE --> SE
CE --> BSC
CE --> BC
BSC --> BS
BC --> BT
CE --> LI
CE --> LC
LC --> LI
LC --> SN
SN --> SL
LI -.webhook reclassificação.-> FS

```

Deploy

```

None
flowchart TB
    subgraph "Produção"
        WS[sharpi-support-automation<br/>FastAPI 1
instância<br/>Render/Fly/Railway]
        WS --> DB[(Postgres<br/>idempotência +<br/>triage_feedback)]
    end

    subgraph "Secrets"
        SM[Secret
Manager<br/>ANTHROPIC_API_KEY<br/>LINEAR_API_KEY<br/>SENTRY_AUTH_TOKEN<br/>BETTERS
TACK_API_TOKEN<br/>BETTERSTACK_SOURCE_ID<br/>BRAINTRUST_API_KEY<br/>WHATSAPP_APP_S
ECRET<br/>WHATSAPP_VERIFY_TOKEN<br/>SLACK_WEBHOOK_URL<br/>ENCORE_INTERNAL_TOKEN]
    end

    WS -.lê.-> SM
    WS -.self-monitoring.-> SE2[Sentry project<br/>support-automation]
    WS -.logs.-> BS2[BetterStack source<br/>support-automation]

```

Dados: schema mínimo

```

None
erDiagram
    PROCESSED_MESSAGES {
        text whatsapp_message_id PK
        text sender_phone
        text group_name
        text linear_issue_id
        text linear_issue_url
        bool was_duplicate
    }

```

```

    text duplicate_of_ticket_id
    timestamp processed_at
}

TRIAGE_FEEDBACK {
    text ticket_id PK
    jsonb original_classification
    jsonb corrected_classification
    text corrected_by
    timestamp corrected_at
    timestamp created_at
}

CLIENT_PROFILE_CACHE {
    text phone_number PK
    text client_id
    jsonb profile_data
    timestamp cached_at
}

```

Taxonomia: 2D interna → flat no display

Internamente a classificação é 2D (**IssueType** × **Component**) pra ter poder analítico. No Linear e no Slack usamos **categoria flat** (igual ao POC visual).

```

None
flowchart LR
    subgraph "2D interno (banco + prompts)"
        C1[product_search]
        C2[approval_flow]
        C3[ai_capture]
        C4[order_launch]
        C5[erp_sync]
        C6[auth]
    end

    subgraph "Flat display (Linear labels + Slack)"
        F1[busca_produtos]
        F2[fluxo_aprovacao]
        F3[captura_ia]
        F4[integracao_erp]
        F5[suporte_geral]
    end

    C1 --> F1

```

```
C2 --> F2
C3 --> F3
C4 --> F4
C5 --> F4
C6 --> F5
```

Função: `component_to_category(component: Component) -> str` em `triage_agent_v2.py`.

Legenda de estados

- **Linha sólida (—):** implementar na Sprint 1
- **Linha tracejada (- -):** Sprint 2 ou depois
- **Feedback loop:** tabela criada na Sprint 1 mesmo que o webhook só popule a partir da Sprint 2 — assim não perdemos dados enquanto o sistema roda em piloto
- **Braintrust + BetterStack:** convivem. Braintrust cobre traces de IA (apenas tickets `ai_capture/ai_failure`); BetterStack cobre logs de infra (todo o resto). Sentry roda em paralelo nos dois caminhos.
- **Retorno ao cliente:** manual na Sprint 1 — suporte lê o ticket e responde o WhatsApp. Outbound automático + CSAT fica pra Sprint 2.